

Efeitos da Prática de Revisão de Código na Caelum: Um Estudo Preliminar em Duas Equipes

Maurício F. Aniche, Francisco Z. Sokol

¹Caelum Ensino e Inovação
Rua Vergueiro, 3185 - 8o andar
São Paulo - SP - Brasil

{mauricio.aniche, francisco.sokol}@caelum.com.br

Abstract. *Code review is how we call the practice of a team member to assess and review the code written by another member. In literature, some studies state that code review helps the team to improve their code quality, as well as reduce the number of defects. In this study, we present a study in two projects at Caelum, a brazilian software development company. By means of mining software repositories techniques and questionnaires, we found that the practice is important to both teams, as it helps people to spread technical knowledge and to reduce defects. In addition, their perception on the code quality is that they are always improving it, even though we did not find any evidence by analyzing their code metrics.*

Resumo. *Revisão de código é o nome dado à prática de um membro da equipe verificar e revisar o código escrito por outro membro. Na literatura, estudos afirmam que as revisões ajudam a equipe a melhorar o código produzido, bem como diminuir a quantidade de defeitos. Neste trabalho, apresentamos um estudo feito em dois projetos da Caelum. Por meio de mineração dos repositórios de código e questionários, descobrimos que a prática de revisão de código é importante para essas equipes, pois as ajudam a disseminar conhecimento e a diminuir a quantidade de defeitos. Além disso, a percepção da equipe sobre a qualidade do código é a de que eles estão sempre a melhorar, apesar de não termos encontrado evidências sobre isso olhando métricas de código.*

1. Introdução

Apesar de não ser uma das práticas mais populares em times ágeis (ela não é nem citada no questionário de 2014 da VersionOne [VersionOne 2014], por exemplo), a revisão de código vem ganhando seu espaço nas equipes de desenvolvimento de software. Afinal, as ditas vantagens da prática são várias: a diminuição na quantidade de defeitos e aumento da qualidade do código produzido, facilitando sua manutenção e evolução. De forma interessante, isso pode ser visto em muitos artigos ao longo dos anos [Siy and Votta 2001, Boehm and Basili 2005, Kemerer and Paulk 2009, Mantyla and Lassenius 2009].

Revisão de código é o nome dado à prática de verificação e validação do código escrito pelos membros da equipe [Kolawa and Huizinga 2007]. Essas revisões podem ser feitas de diversas maneiras, como, por exemplo, programação pareada, ou mesmo navegando pelos artefatos modificados. Comumente, o revisor anota todos os problemas en-

contrados e devolve ao autor original daquele código. O autor então avalia os comentários recebidos e eventualmente os propaga para o código-fonte.

Na Caelum, em particular, as equipes fazem uso do próprio Github¹, o serviço de hospedagem de código, para fazer suas revisões. Após a finalização de uma história, o desenvolvedor anota a lista de *commits* e artefatos modificados. Um outro membro do time, em posse dessa lista, inspeciona todo o código modificado; todo e qualquer problema encontrado pelo revisor é salvo em comentários no próprio Github. O desenvolvedor original, ao receber os e-mails enviados automaticamente pela ferramenta, discute e tira dúvidas com seu revisor, e eventualmente altera o código para satisfazer a sugestão. Na Figura 1, mostramos um exemplo de comentário feito no Github.

Neste trabalho, relatamos a experiência de duas equipes, ambas da Caelum, com a prática de revisão de código. O objetivo do estudo é entender as reais razões pelas quais as equipes da Caelum fazem uso desta prática, e descobrir se as vantagens que a literatura defende também acontecem nessas equipes.

Ao final do trabalho, percebemos que ambas as equipes se beneficiam da disseminação de conhecimento que acontece durante a revisão e da redução de defeitos, já que revisores geralmente encontram problemas na implementação original. Além disso, a percepção dos desenvolvedores é a de que a qualidade do código aumenta. Entretanto, não detectamos essa melhora olhando para métricas de código.

Este artigo está dividido da seguinte maneira: na Seção 2, citamos alguns trabalhos relacionados à revisão de código; na Seção 3, descrevemos todo o estudo, planejamento, equipes e projetos, questionários e métricas de código utilizados; na Seção 4, apresentamos o resultado dos estudos quantitativos e qualitativos feitos, por meio de gráficos e trechos dos questionários; na Seção 5, discutimos os resultados encontrados; na Seção 6 apontamos para algumas possíveis ameaças à validade do estudo; e, por fim, na Seção 7, deixamos nossas conclusões e trabalhos futuros.

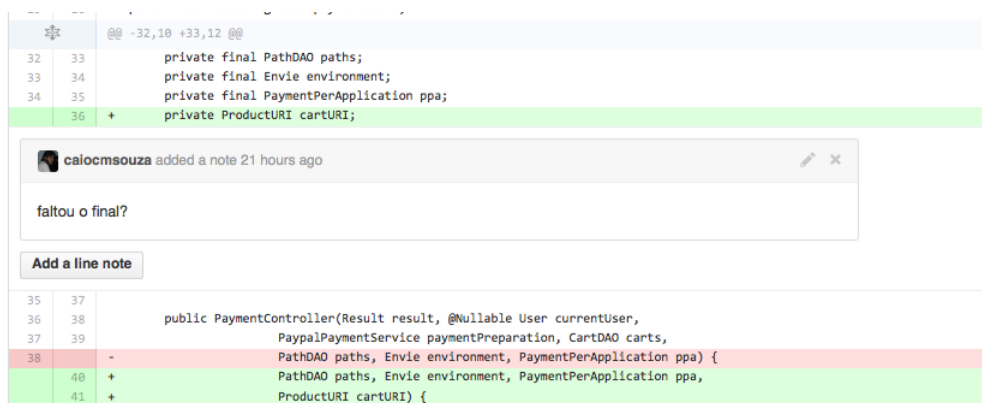


Figura 1. Exemplo de uma revisão de código feita por meio de comentários no Github

2. Trabalhos relacionados

Alguns estudos publicados avaliaram os efeitos da prática de revisão de código sobre diferentes aspectos. Em [Kemerer and Paulk 2009], os autores realizaram uma pesquisa

¹<http://www.github.com>. Último acesso em 26 de agosto de 2014.

quantitativa sobre dados de diversos projetos que seguiam um processo formal de revisão de código. Com base nos dados analisados, os autores concluíram que revisão de código é uma prática eficaz para localizar defeitos (na definição dos autores, um defeito causa uma falha no sistema em tempo de execução). Além disso, o estudo destaca que a taxa ideal de revisão de código é 200 linhas por hora por desenvolvedor, de forma que se esse ritmo for mais baixo, a revisão de código pode atrasar o projeto; já se for maior, torna-se menos efetiva na localização de defeitos.

Já em [Siy and Votta 2001], os autores relatam um estudo realizado em projeto de software que também seguia um processo formal de revisão de código. Os autores mostram que a maioria dos defeitos encontrados durante a revisão de código é leve, isso é, não afeta o comportamento esperado da execução do programa. Apesar disso, o estudo demonstra que, ao corrigir esses pequenos defeitos, o código ganha em manutenibilidade e por isso a prática de revisão de código deve ser valorizada.

Confirmando o estudo citado anteriormente, em [Mantyla and Lassenius 2009], os autores também mostram que a prática de revisão de código, na maior parte das vezes, encontra defeitos não-funcionais. O trabalho relata dois estudos empíricos sobre revisão de código, um realizado sobre um projeto na indústria e outro com um experimento com alunos de graduação. Os autores se dedicam a classificar detalhadamente os tipos de defeitos encontrados no processo de revisão e mostram que 77% dos defeitos são não-funcionais na indústria, enquanto 85% é deste tipo nas revisões feitas por estudantes.

3. Estudo

Nesta seção, apresentamos o planejamento do estudo, a descrição dos projetos e equipes que participaram, bem como os dados, métricas e questionários utilizados ao longo dele. Na Subseção 3.1, discutimos o planejamento e a condução do estudo; na Subseção 3.2, descrevemos as equipes e projetos avaliados; na Subseção 3.3, explicamos as métricas de código utilizadas no estudo quantitativo; e na Subseção 3.4, descrevemos o questionário usado para extrair as opiniões dos membros da equipe.

3.1. Planejamento do estudo

Tínhamos à disposição duas equipes que trabalhavam em projetos diferentes dentro da Caelum. Como dito anteriormente, ambas as equipes fazem revisão de código no Github. Esse serviço possibilita ao desenvolvedor escrever comentários em qualquer linha, em qualquer artefato, e em qualquer *commit*. Depois de implementar uma funcionalidade e consolidar a alteração no Github, o desenvolvedor passa o *link* do *commit* para outro integrante da equipe, que escreve comentários sobre o código desenvolvido.

Em um primeiro momento, optamos por avaliar se a qualidade interna do código melhorava após a revisão de código. Para isso, optamos por minerar o repositório de código e extrair essa informação do Github. Durante as análises, consideramos apenas dados entre agosto de 2013 (momento em que as equipes começaram a fazer a revisão de código pelo Github) e agosto de 2014 (momento atual da escrita do trabalho).

O Github disponibiliza todas as informações armazenadas em seus projetos por meio de uma API. Nós então desenvolvemos uma simples aplicação que extrai todos os comentários feitos em todos os *commits* do projeto. Por meio dela, armazenamos as

informações mais importantes para esse estudo, que eram o nome do autor, a data do comentário e o arquivo comentado. Também executamos o MetricMiner [Sokol et al. 2013], uma ferramenta de código aberto que apoia a extração de métricas de código, também desenvolvida pelo nosso grupo de pesquisa. A ferramenta é capaz de extrair diversas métricas de código ao longo do histórico do repositório.

Com o conjunto de métricas de código e a relação de comentários feitos, nós relacionamos um ao outro: separamos métricas de código de um artefato **exatamente antes** e **exatamente depois** de uma revisão. Para isso, pegamos os *commits* com datas imediatamente antes e imediatamente depois da data do comentário.

Verificamos, então, a diferença entre os números. Suponha, por exemplo, uma métrica de quantidade de linhas de código: se seu valor for menor após a revisão de código, isso significa que a classe diminuiu de tamanho após a revisão. Nesse estudo, olhamos para várias métricas de código convencionais. Elas estão listadas na Seção 3.3.

Com esses números em mãos, mostramos os dados aos membros das equipes para que os interpretassem. Para isso, usamos um questionário aberto, detalhado na Seção 3.4, no qual eles descreveram por que fazem revisão de código e, pelo ponto de vista deles, quais as vantagens da prática. Mas, para não enviesá-los com os dados encontrados, dividimos o questionário em duas etapas: na primeira, perguntamos sua opinião pessoal sobre a prática, e na segunda, que acontecia após ele ter visto os gráficos, pedíamos para ele interpretar e nos dizer o porquê daqueles números.

Todos os dados, como as métricas de código, comentários e as respostas dos questionários, estão abertos e podem ser encontrados no *Google Drive*².

3.2. As equipes e projetos

Tínhamos à disposição duas equipes diferentes que trabalhavam em dois projetos diferentes. Os projetos, conhecidos internamente por *Caelumweb* e *Gnarus*, são aplicações web desenvolvidas em Java, e fazem uso das mesmas decisões técnicas: usam VRaptor como *framework* MVC, Hibernate para acesso a banco de dados, JSPs e Taglibs para camada de visualização.

Ambos os projetos são de médio porte, tendo por volta de 1000 classes e aproximadamente 10 mil commits. O *Caelumweb*, que é um projeto mais velho, tem por volta de oito anos de vida. Já o *Gnarus* vem sendo desenvolvido pelos últimos três anos. Na Tabela 1, mostramos o tamanho dos projetos.

Atualmente, a equipe do *Gnarus* conta com quatro desenvolvedores e o *Caelumweb* com sete desenvolvedores fixos. Nenhum membro das equipes já participou de algum treinamento sobre revisão de código; não há regras a serem seguidas. Eles apenas leem o código e comentam de acordo com o que acham necessário.

3.3. Métricas de código

As métricas de código que escolhemos para este estudo foram as já consolidadas pela indústria e pela academia. Elas também foram escolhidas por comodidade, já que são as métricas já implementadas no MetricMiner. Dado que ambos os projetos são escritos em

²O link encurtado para a pasta no *Google Drive* é <http://bit.ly/1tOMRrG>. Último acesso em 28 de agosto de 2014.

Tabela 1. Tamanho dos dois projetos analisados

Projeto	# de classes	# de <i>commits</i>	# de autores diferentes
Gnarus	924	10451	33
Caelumweb	1321	12077	59

Java, que é uma linguagem orientada a objetos, temos métricas para três diferentes pontos de vista: complexidade (complexidade ciclomática, quantidade de linhas de código, quantidade de métodos), coesão (*LCOM-HS*) e acoplamento (*Fan-Out*).

- **Complexidade ciclomática (CC):** a métrica nos diz quantos caminhos diferentes um único método pode ter. A implementação utilizada aqui é o *número de McCabe*. Como a métrica é geralmente calculada por método individual, somamos os números de todos os métodos para calcular a complexidade ciclomática da classe.
- **Acoplamento eferente (*Fan Out*):** a métrica conta a quantidade de classes de que a classe original depende. Como a ferramenta faz uso de análise estática, ela apenas conta a quantidade de classes importadas no começo de cada arquivo Java.
- **Falta de coesão dos métodos (*LCOM-HS*):** a métrica nos diz o quanto uma classe é ou não coesa. O algoritmo mais conhecido é o *Handerson-Sellers*, e é o utilizado pela ferramenta.
- **Quantidade de linhas de código:** A métrica conta a quantidade de linhas de código por classe. Ela não faz nenhum tipo de filtro, ou seja, ela conta linhas em branco, linhas com apenas abre ou fecha chaves, e etc.
- **Quantidade de métodos:** a métrica conta a quantidade de métodos por classe, independente do modificador de acesso desse método.

Em todas as métricas escolhidas, *quanto "maior" o valor, "pior" é a classe*. Ou seja, quanto maior o valor da complexidade ciclomática, mais complicada aquela classe é.

3.4. Questionário

O questionário entregue aos desenvolvedores estava dividido em duas etapas. A versão original pode ser encontrada no *Google Forms*³.

Em primeiro lugar, perguntamos ao sujeito quais vantagens ele via na revisão de código em seu projeto, bem como seus dados para futuro contato:

1. Seu nome?
2. Seu projeto?
3. Na sua opinião, quais as vantagens de se fazer revisão de código no seu projeto?
Quais são os benefícios? Por favor, elabore bem sua resposta.

Em seguida, mostramos os gráficos e os ensinamos a avaliá-los. Ficou claro para todos eles que, do ponto de vista das métricas de código, a qualidade interna não melhorava (os números são detalhados na próxima seção).

Na sequência, eles responderam a mais um conjunto de perguntas. O objetivo era ver se eles mudavam de ideia, ou se surpreendiam com os dados encontrados:

³A URL encurtada para o formulário é <http://bit.ly/1scXO5Y>

1. Na sua opinião, por que os números não melhoraram mesmo após a revisão de código? Por favor, elabore bem sua resposta.
2. Sabendo disso, você acha justo parar de fazer revisão de código no seu projeto? Por favor, elabore bem sua resposta.

Para a análise do questionário, não utilizamos nenhuma técnica específica de codificação, como é recomendado pela Teoria Fundamentada nos Dados (em inglês, *Grounded Theory*). Ambos os pesquisadores leram de maneira individual cada uma das respostas, as interpretaram e, juntos, chegaram às conclusões aqui relatadas.

4. Resultados encontrados

Nas Figuras 2 e 3, encontradas ao final deste artigo, mostramos a distribuição das métricas calculadas nos projetos. Como temos em mãos os valores das métricas antes e depois da revisão, subtraímos uma da outra, e traçamos o histograma desses números. O valor zero no gráfico significa que determinada métrica de código não mudou após a revisão de código; um valor negativo indica que ela piorou; por fim, o valor positivo indica que ela melhorou após a revisão.

Olhando para os gráficos, percebemos que, na grande maioria dos casos, as métricas não são influenciadas pela revisão. Em todas elas, aparecem casos em que as métricas melhoram ou pioram, mas elas são minoria. **Ou seja, concluímos que, em ambos os projetos, a revisão de código não tem efeito positivo nos valores das métricas de código calculadas.**

Já no questionário, em sua pergunta inicial, os desenvolvedores deixaram claro que há três principais motivos pelos quais eles acreditam que a revisão de código os ajuda: melhoria da qualidade interna do código, diminuição na quantidade de defeitos e a disseminação de conhecimento inerente ao processo. Na caixa a seguir, mostramos trechos dos questionários:

“Alguns desenvolvedores aprendem técnicas e tecnologias com os outros.”

“Garante melhor qualidade no código, além de gerar umas boas discussões sobre como foi feito e no que pode melhorar.”

“Ensino de boas práticas para pessoas menos experientes, mais clareza e ciência dos padrões de código do projeto, bugs são pegos mais cedo...”

“Evitam nomes confusos e ajudam a deixar um código mais claro e de manutenção mais fácil. (...)”

Um membro da equipe até alertou para o fato de que a equipe ainda tem muitos desenvolvedores iniciantes e que ainda precisam aprender mais sobre boas práticas de codificação. E, para ele, a revisão de código os ajuda nisso.

Mas, na segunda parte do questionário, após verem os números e gráficos, nos quais aparentemente as revisões não ajudam na qualidade interna do código, as respostas se focaram mais na ideia de encontrar possíveis defeitos na implementação. Segundo eles, a revisão de código ajuda os desenvolvedores a encontrar possíveis caminhos excepcionais, ou mesmo regras de negócio que não foram implementadas pelo desenvolvedor original:

“(...) menos bugs entram em produção, já que todo código é revisado... para passar algo, duas pessoas precisam “errar” é não uma.”

“(o desenvolvedor) acha erros onde a pessoa não está acostumada a olhar (...)”

“(...) a maioria das coisas que pegamos no code review são possíveis bugs ligados à regra de negócio.”

“Uma revisão, não necessariamente deve melhorar a qualidade do código. É mais pra garantir que o código vai funcionar como deve (inclusive sendo “mantível” no futuro).”

Um desenvolvedor, em particular, sugeriu que as mudanças feitas no código são, em sua maioria, focadas em legibilidade e, portanto, as métricas escolhidas não pegariam esse tipo de revisão:

“Tenho a impressão de que, em geral, nós (ou eu, pelo menos) não olhamos para essas métricas exatamente para revisar um código. O que eu olho é se tem algum erro na lógica, algum jeito de deixar o código mais fácil de entender, algum detalhe que faltou etc.”

Justamente por esses motivos, ao perguntarmos se eles cogitariam a ideia de abandonar a prática de revisão de código no projeto, a resposta foi unânime: não. E, na opinião deles, o aprendizado ao ler o código do colega, junto com a diminuição de defeitos são motivos suficientes para que a equipe continue com a prática:

“Não acho justo. Eu pelo menos tenho muito a ganhar nas revisões, seja quando eu estou revisando ou quando alguém está revisando meu código. Acredito que este tipo de coisa não pode ser mensurada.”

“Não, já que eu sinto uma melhora, principalmente na quantidade de bugs que sofríamos ligados à regra de negócio.”

“Ainda acredito que a prática de revisão é importante, ajudando-nos a encontrar possíveis erros já vivenciados por outros desenvolvedores. O code review pode ajudar muito também no processo de aprendizado de um novo membro da equipe.”

Um detalhe relatado por uma desenvolvedora mais nova no projeto é de que talvez a equipe não esteja praticando revisão de código da melhor forma possível. De fato, estudos recomendam que desenvolvedores revisem no máximo 200 linhas de código por hora [Mantyla and Lassenius 2009], o que não é seguido pela equipe:

“Eu estou há pouco tempo fazendo revisão de código, mas pelo que pude perceber, na prática, as revisões se acumulam e depois é preciso fazer um grande número de uma vez só. Além disso, enquanto umas pessoas se atentam mais a detalhes durante a revisão, outras passam despercebidas por vários pontos. Se os números não melhoram diante da revisão de código, talvez podemos estar perdendo tempo

com essa prática. Ou talvez fazemos de uma forma errada, não sei se isso é possível. Então, pode ser justo parar ou achar alguma alternativa.”

5. Discussão

Pelos dados apresentados na seção anterior, podemos concluir que ambas as equipes estudadas têm benefícios ao praticar revisão de código. Mais precisamente, a disseminação de conhecimento e a redução de defeitos são os efeitos notados pelo time.

Ao ler o código de outro desenvolvedor, o revisor acaba por aprender boas práticas de codificação, diferentes técnicas de implementação de algoritmos, e até mesmo funcionalidades da linguagem e dos *frameworks* utilizados. De maneira análoga, ao ler o código, o revisor também encontra possíveis caminhos não tratados, ou mesmo implementações errôneas feitas pelo desenvolvedor. A consequência disso é a diminuição de defeitos.

Quanto à qualidade interna, apesar dos desenvolvedores também acreditarem que a revisão os ajuda nisso, isso não apareceu aos olhos das métricas de código. Mas, como alertado anteriormente por um participante, muitas das revisões são focadas em legibilidade de código, um fator que não é capturado por nenhuma das métricas utilizadas.

Acreditamos também que, embora o código não melhore diretamente pela revisão do código, uma vez que os desenvolvedores estão constantemente aprendendo com o código dos outros, isso faz com que seus próximos códigos já incorporem as boas práticas que aprenderam com as revisões anteriores.

Na Tabela 2, mostramos uma comparação entre os trabalhos relacionados anteriormente e os resultados desse estudo. Nosso estudo, de certa forma, corrobora com os resultados que já eram conhecidos.

	Diminuição de Defeitos	Melhora na qualidade interna	Disseminação de conhecimento
Kemerer et al	X		
Siy et al		X	
Mantyla et al	X	X	X
Este estudo	X	X ⁴	X

Tabela 2. Comparação entre trabalhos da literatura e este estudo

Portanto, concluímos que ambas as equipes se beneficiam da disseminação de conhecimento e da redução de defeitos, consequências da prática de revisão de código. As equipes também têm uma percepção de melhoria da qualidade do código.

6. Ameaças à validade

Nesta seção, listamos algumas possíveis ameaças à validade deste estudo.

⁴Em nosso estudo, a qualidade interna foi observada apenas na opinião pessoal dos desenvolvedores. As métricas de código não apontaram melhorias.

6.1. Validade interna

- A heurística utilizada para gerar os gráficos pode ter problemas. Ou seja, olhar o valor da métrica exatamente antes e exatamente depois da revisão pode não ser preciso. Por vezes, a revisão de código demora a acontecer, e a classe pode ter sido alterada nesse tempo. Mas, frente aos dados encontrados na análise qualitativa, acreditamos que isso não afeta a discussão feita aqui.
- Apesar de os desenvolvedores afirmarem que o aprendizado é intenso e que a quantidade de defeitos diminuiu, não triangulamos essas informações e, portanto, elas representam o ponto de vista deles, que pode ser enviesado.

6.2. Validade externa

- A maneira informal que é feita e a falta de treinamento específico em revisão de código podem diferir em outras empresas.

7. Conclusão e futuros trabalhos

Neste trabalho, relatamos a experiência de duas equipes da Caelum com a prática de revisão de código. Os resultados mostram que ambas as equipes se beneficiam da inerente disseminação de conhecimento que acontece ao ler o código produzido por outro desenvolvedor, e da redução de defeitos, já que o revisor comumente encontra problemas no código produzido pelo desenvolvedor original. Os desenvolvedores também têm a percepção de melhoria da qualidade interna, mas isso não se refletiu nas métricas de código coletadas. Os resultados corroboram com estudos anteriores na área.

Neste passo do trabalho, não temos a pretensão de generalizá-lo para outras equipes. Mas acreditamos que os dados encontrados aqui façam com que as equipes de desenvolvimento de software pensem melhor sobre por que e como estão usando a prática de revisão de código.

Como lição, pretendemos ministrar treinamentos internos sobre como executar revisões de código, de forma a obter melhores *feedback* também sobre a qualidade interna do código. E, assim que feito, pretendemos re-executar o estudo e ver se os resultados encontrados aqui mudam.

Sugerimos que outras empresas repitam o mesmo estudo e questionário, para que elas também descubram quais são os reais efeitos da prática em suas equipes. Além disso, melhorar os instrumentos, como o questionário e a heurística para identificar os *commits* de melhoria podem trazer uma nova perspectiva para este estudo.

Agradecimentos

Agradecemos à Caelum Ensino e Inovação e aos seus desenvolvedores por participarem do estudo.

Referências

- Boehm, B. and Basili, V. R. (2005). Software defect reduction top 10 list. *Foundations of empirical software engineering: the legacy of Victor R. Basili*, 426.
- Kemerer, C. F. and Paulk, M. C. (2009). The impact of design and code reviews on software quality: An empirical study based on psp data. *Software Engineering, IEEE Transactions on*, 35(4):534–550.

- Kolawa, A. and Huizinga, D. (2007). *Automated Defect Prevention: Best Practices in Software Management*. Wiley.
- Mantyla, M. V. and Lassenius, C. (2009). What types of defects are really discovered in code reviews? *Software Engineering, IEEE Transactions on*, 35(3):430–448.
- Siy, H. and Votta, L. (2001). Does the modern code inspection have value? In *Proceedings of the IEEE international Conference on Software Maintenance (ICSM'01)*, page 281. IEEE Computer Society.
- Sokol, F. Z., Aniche, M. F., and Gerosa, M. A. (2013). Metricminer: Supporting researchers in mining software repositories. In *Source Code Analysis and Manipulation (SCAM), 2013 IEEE 13th International Working Conference on*, pages 142–146. IEEE.
- VersionOne (2014). 8th annual state of agile survey.

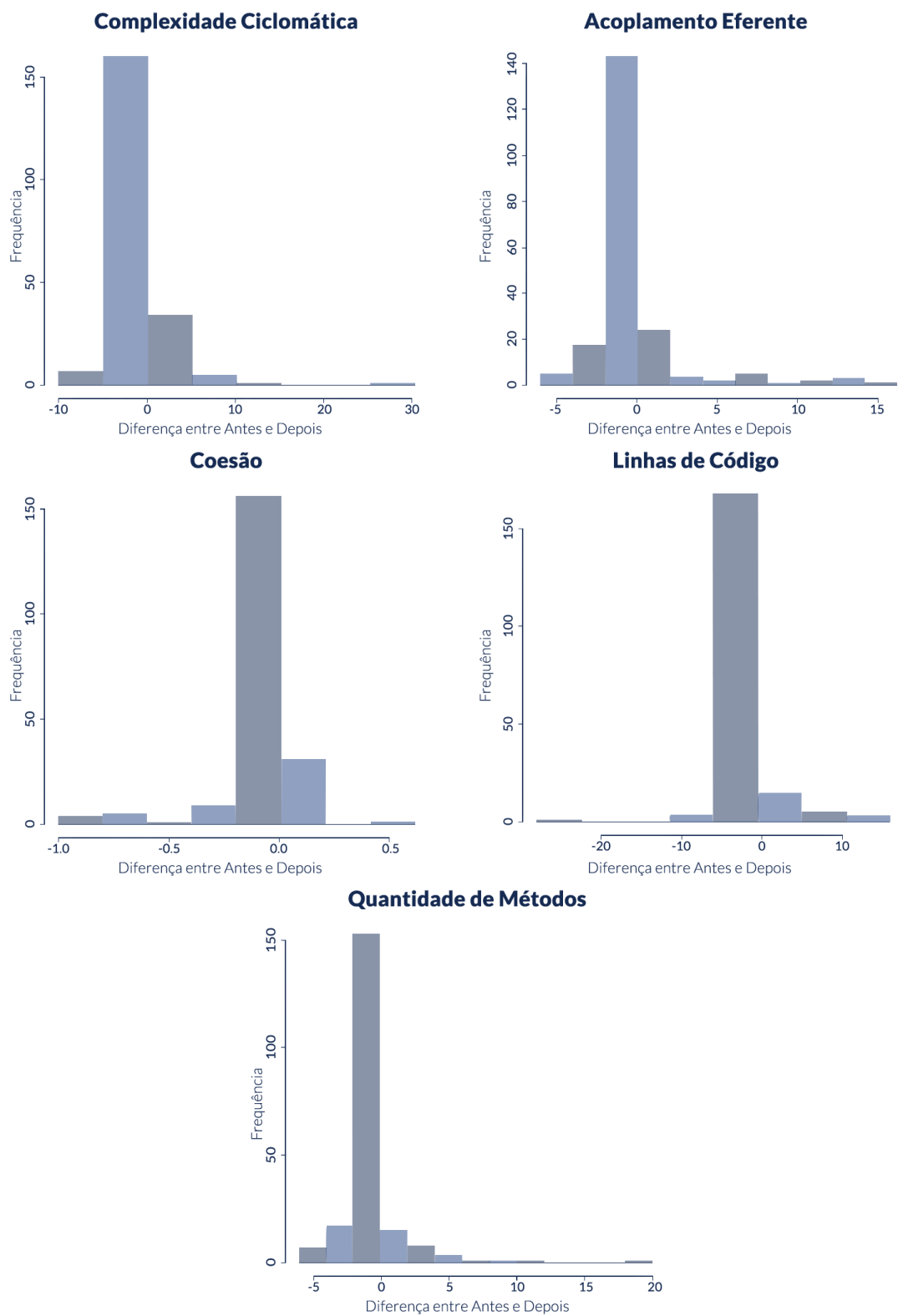


Figura 2. Distribuição das métricas no *Caelumweb*

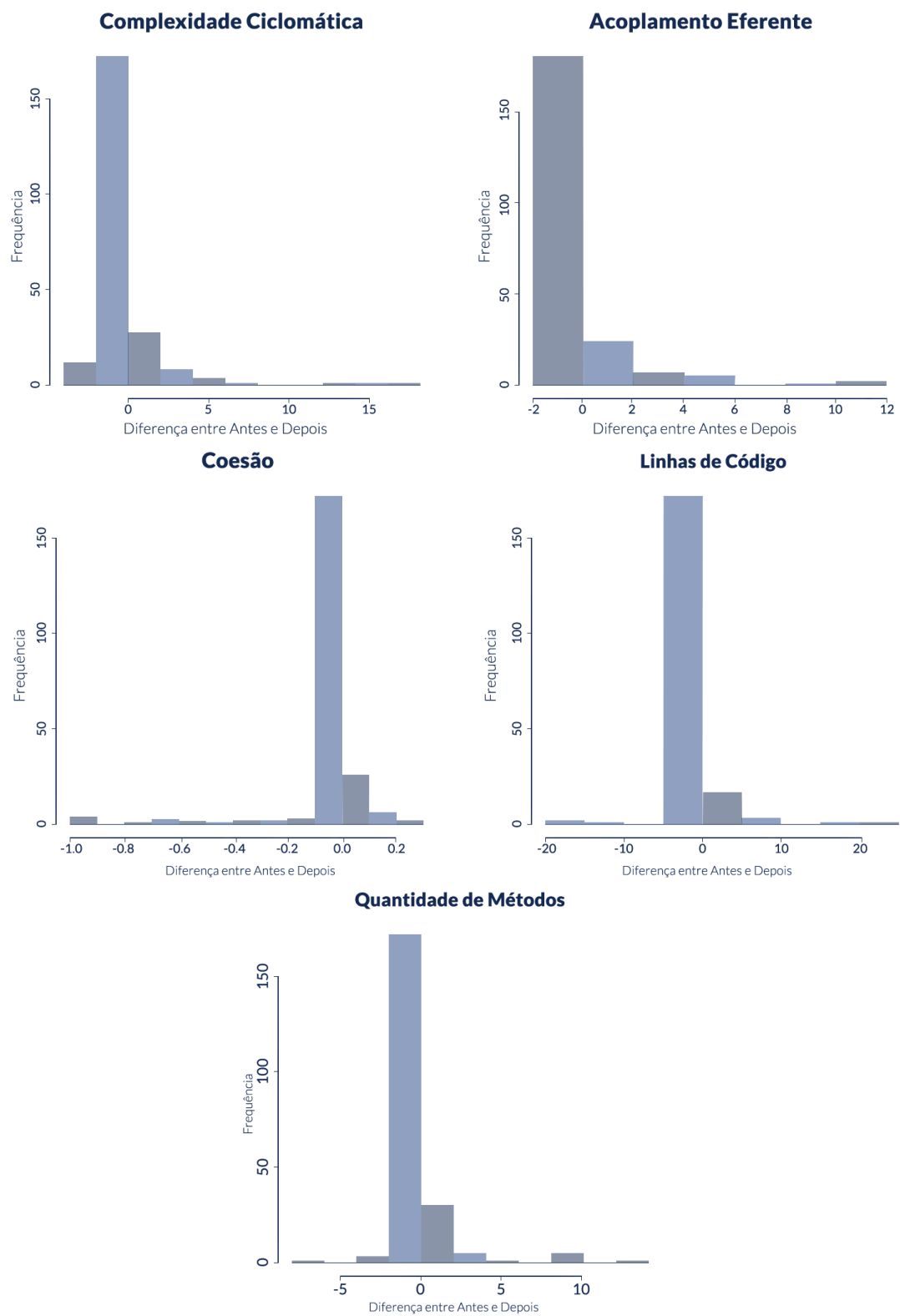


Figura 3. Distribuição das métricas no *Gnarus*